UC San Diego

Generating Phylogenetic Trees Using MASH and Neighbor Joining Algorithm

Aditya Kamat | Devansh Gupta

Overview and Motivation

- Core Challenge: Determine genomic similarity through a fast and scalable approach
- Mash and Jaccard Index to measure similarity between pair of sequences

$$D = -\frac{1}{k}\ln\left(\frac{2J}{1+J}\right)$$

- Neighbor Joining
- Advantages over other methods

	Multiple Sequence Alignment	**MASH Algorithm**
Time Complexity	$O(N^2L^2)$	$O(NL)$
Memory Complexity	$O(N^2L^2)$	$O(NS + N^2)$

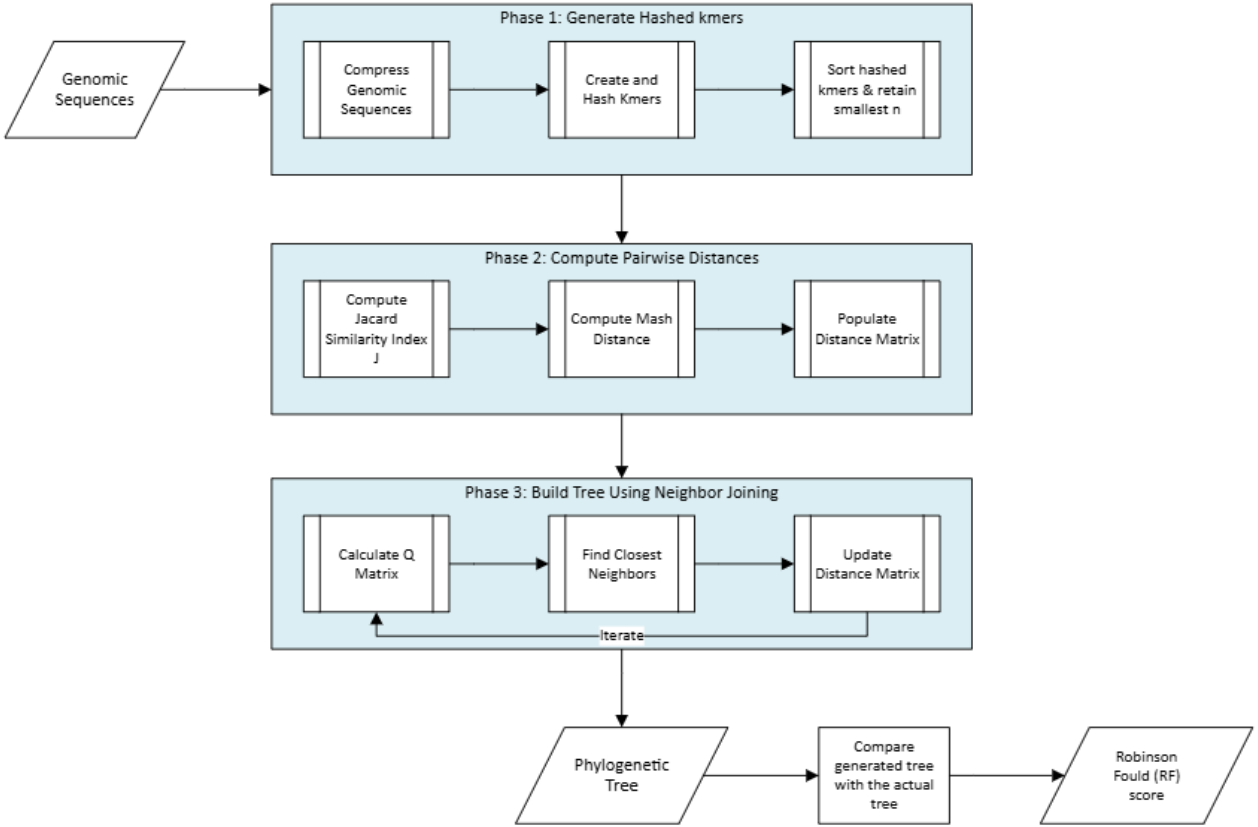
N -> Number of sequences; L -> Length of each sequence; S -> Sketch size

Prior Work and Current SOTA

UC San Diego

- **Dashing**: Uses alternating Hashing Strategies (HyperLogLog)
- **SKmer**: Error Corrected Sketching (SKetching genomes from MERs)
- **DIPPER**: Utilizes different techniques (like placement or divide and conquer) to generate phylogeny based on number of sequences
- Baseline Implementation:
  - Developed our own sequential baseline code to generate phylogenetic tree
  - Use minHash and Neighbor Joining Algorithms
  - Verified the baseline by comparing the tree with the ground truth tree available from DIPPER

Methodology



Parallelization Approach

UC San Diego

- Generate Mash sketches (for all sequences): map parallelism across sequences/blocks.
  - Compress the genomic sequence with binary encoding - map parallelism across threads
  - Create kmers and hash them using minHash - map parallelism
  - Sort the hashed kmers and retain smallest n hashes - parallel sort

- Compute pairwise distances between sketches As and Bs: Map parallelism across GPU Blocks (pair of sequences assigned to one block)
  - Compute Jaccard similarity index $J = |As \cap Bs| / |As \cup Bs|$ - Map parallelism across GPU Blocks
  - Compute Mash distance from Jaccard index - Map parallelism across threads in a block
  - Populate distance matrix storing pairwise genomic similarity score

- Build a tree using the Neighbor-Joining algorithm:
  - Calculate the Q-matrix - Map parallelism
  - Find closest neighbors - Parallel min-reduction
  - Update the distance matrix - Map parallelism

Evaluation metrics

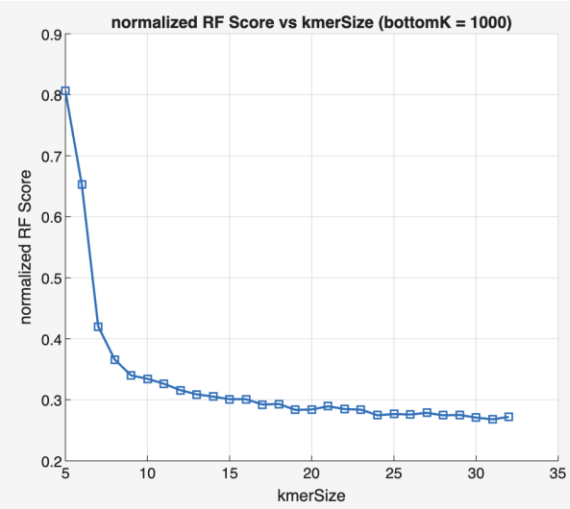
UC San Diego

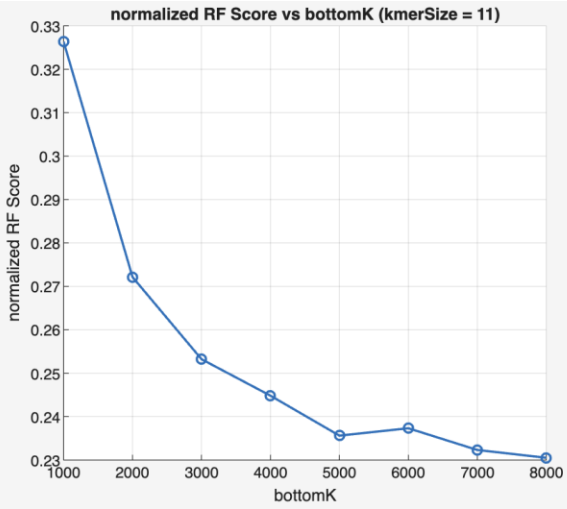
- Used DIPPER dataset having 10000 genomic sequences to compute the phylogenetic tree
- Compared baseline sequential code to compute speedup achieved for each kernel
- Compared the generated trees with the reference tree from the DIPPER dataset. The normalized Robinson Fould score is used as a metric to quantify the accuracy of our generated trees
- Analyze impact of varying kmerSize and bottomK on the accuracy of the generated trees
- Analyze the impact of varying bottomK on the runtime

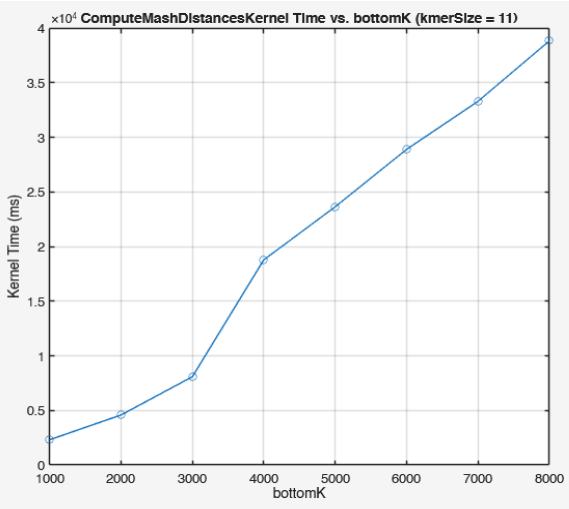
Results

GPU Kernel Execution Time: Sequential vs. Parallel (kmerSize = 11, bottomK = 8000)					
1,000 sequences | 49,995,000 pairs | NVIDIA A30 MIG 2g.12gb					
Kernel	**Sequential Total (ns)**	**Parallel Total (ns)**	**Sequential Total (ms)**	**Parallel Total (ms)**	**Speedup**
ComputeMashDistancesKernel	639,348,354,311	217,173,979	639,348.354	217.174	**2943.95x**
FindMinQ + FinalizeTopology	23,273,376,567	218,445,715	23,273.377	218.446	**106.54x**
ComputeNetDivergence_Kernel	20,224,370,357	72,911,274	20,224.370	72.911	**277.38x**
HashAllKmersKernel	4,251,872,296	3,948,922	4,251.872	3.949	**1076.72x**
kmerArrCreate	542,113,006	5,163,873	542.113	5.164	**104.98x**
UpdateDistanceMatrix_Kernel	148,602,157	2,437,754	148.602	2.438	**60.96x**
InitNJMatrixKernel	39,851,588	44,416	39.852	0.044	**897.23x**
TOTAL	**687,828,540,282**	**520,125,933**	**687,828.540**	**520.126**	**1322.43x**

Results


normalized RF Score vs kmerSize (bottomK = 1000)
normalized RF Score
kmerSize
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
5
10
15
20
25
30
35


normalized RF Score vs bottomK (kmerSize = 11)
normalized RF Score
bottomK
0.33
0.32
0.31
0.3
0.29
0.28
0.27
0.26
0.25
0.24
0.23
1000
2000
3000
4000
5000
6000
7000
8000


×10^4 ComputeMashDistancesKernel Time vs. bottomK (kmerSize = 11)
Kernel Time (ms)
bottomK
4
3.5
3
2.5
2
1.5
1
0.5
0
1000
2000
3000
4000
5000
6000
7000
8000

UC San Diego

Thank you!